AI-Augmented "Prompt Log"

Backend Edition · Guardian Integration Gateway

Candidate Name	Jorge Alcazar Diaz
Role	Senior Backend Engineer (AI & Integration)
Tool(s) Used	Claude Code (Anthropic CLI) running Claude Opus 5.5, with sub-agents for focused roles (implementation writers, four independent read-only code reviewers, a refuter, a scoped fix validator). Verification with Vitest, ESLint, the TypeScript compiler, and Docker.
Repository	github.com/alcazar9491/guardian-integration-gateway
Demo video	loom.com/share/d420f65af28a42b6aadbd50f88f8eae8

1. The Strategy (Architecture & Data Flow)

I planned a REST service (Express 5, strict TypeScript) in Clean Architecture: the domain and application layers never touch Express, the filesystem, or environment variables, and an ESLint allow-list enforces it. The non-negotiable rule of the data flow: **no text leaves the service before it is redacted, and the original message only ever exists encrypted.**

```
POST /secure-inquiry
  → thin controller: JSON + UTF-8 only, zod schema (strict, size limits)
  → ProcessSecureInquiry use case
      1. sanitize message  → RedactedText (branded type: raw strings cannot reach the provider)
      2. encrypt original  → AES-256-GCM, AAD = [auditId, userRef]
      3. audit revision 1 (PENDING)   → if this write fails, the provider is never called
      4. CircuitBreaker → Timeout → MockAiProvider (2 s)
      5. audit revision 2 (outcome)
  → presenter → consistent JSON (COMPLETED | DEGRADED + reason)
  OPEN circuit: step 4 rejects synchronously, without calling the provider → "Service Busy" in milliseconds.
```

Before generating any code, I ran a design-only prompt (step 01) to stress-test this model: requirements, security assumptions, ports, patterns, race conditions, and API contracts.

2. The Prompt Chain (Execution)

Prompts abbreviated to their opening and key constraints; the full verbatim prompts are kept in the session's prompt list.

Step	Your Prompt (What you asked)	AI Response Analysis (What it gave you)	Action Taken
01	*"Act as a Senior Backend Engineer and Software Architect… Analyze the assignment… requirements, security assumptions, Clean Architecture structure, interfaces, SOLID, patterns, race conditions, API contracts, 4–6 h plan. Do not implement anything yet."*	Requirements, layer layout, ports, patterns, and a breaker transition table. Strong proposals: JSON Lines instead of rewriting a JSON array (which loses records under concurrency); audit *before* the provider call (fail-closed); a branded `RedactedText` type so raw text cannot reach the provider at compile time.	Accepted as the baseline with 14 documented assumptions. Refined later: the exact "Service Busy" text and storing the outcome as two audit revisions.
02	*"Implement the initial project structure… strict TypeScript… separate layers… config, centralized error handling, structured logging, validation… DI via composition root… business rules independent of Express, fs, env… only necessary dependencies."*	A walking skeleton (stubs returning 501) and ESLint layer rules proven with deliberate violations. Good catch: body-parser errors carry the raw request body and V8 JSON errors echo input, so they must never be logged. Bad: it "fixed" a failing `charset=utf-7` test by **changing the test input** instead of investigating, and it held the AES key as a raw `Buffer` in config.	Accepted the structure; flagged the masked test as unresolved and root-caused it in step 08. The `Buffer` key was caught in step 04.

Step	Your Prompt (What you asked)	AI Response Analysis (What it gave you)	Action Taken
03	"*Implement a robust PII sanitizer… realistic card lengths with boundaries… SSN + standalone 9-digit… exact labels… overlapping matches, no double-redaction… deterministic precedence rule… no logging of sensitive values…*"	Detection → resolution pipeline: independent detectors scan the *original* text, overlaps merge into one redaction labeled by the longest candidate (tie: EMAIL > CREDIT_CARD > SSN), one rewrite pass, idempotent; 200 tests. It amended its own rule after finding that "commas join numbers" leaked `123456789,987654321`, and documented two edge cases of the strict boundary rule.	Accepted; required every trade-off to be documented. Kept the strict boundary rule because the requirement forbids treating arbitrary long numbers as cards.
04	"*Implement an encryption service… AES-256-GCM… unique CSPRNG IV… 32-byte key from env, validated at startup… store only ciphertext, IV, tag, metadata… decryption for tests/authorized verification only… never log secrets.*"	Random 12-byte IV, 16-byte tag, AAD binding to the record. Found a real leak: `JSON.stringify(config)` printed the key bytes (`Buffer` serialization). Its first leak test passed **against** the bug (it only searched for base64/hex). It confirmed on Node 24.19 that a decipher without `authTagLength` accepted a **forged 4-byte tag**.	Required the leak test to fail against the old code before accepting it. Key → opaque `KeyObject` with the decoded buffer zeroed; tag pinned to 16 bytes; separate encrypt/decrypt ports so the request path cannot decrypt.
05	"*Implement an AIProvider interface and a MockAIProvider… 2 s async delay… deterministic failure injection… consecutive failures then recovery… never receives unsanitized text… typed errors… cancellation/timeout.*"	Replaced the scaffold's **random** failure rate with deterministic schedules (`failFirst(n)`, `scripted`), an abortable non-blocking delay, a timeout decorator, and typed errors. Caller cancellation is explicitly **not** counted as a breaker failure. Added an ESLint rule forbidding `as RedactedText` outside the sanitizer.	Accepted; rejected randomness as incompatible with deterministic tests.
06	"*Implement a production-minded CircuitBreaker… CLOSED/OPEN/HALF_OPEN… exact 3-failure threshold… reject immediately while OPEN… controlled HALF_OPEN… safe under concurrency… injectable clock… define whether the 3rd failing request returns an error or a fallback.*"	Lazy transitions (no timers), synchronous fail-fast, a generation counter against stale outcomes, a single probe. Defined the 3rd failing request as a `PROVIDER_ERROR` fallback (it did reach the provider); only later requests get `CIRCUIT_OPEN`. Measured: 4th call under 50 ms with a 2 s provider whose delay ran exactly 3 times.	Accepted; analyzed line by line in Section 3.
07	"*Implement an AuditRepository and a JSON-file adapter… ID, timestamp, userId under a minimization policy, encrypted original, sanitized text, outcome… concurrent-write safety… safe file writes… document limits.*"	Reconciled "store the outcome" with fail-closed auditing through two append-only revisions (`PENDING` before egress, outcome after). Replaced the raw `userId` with an HMAC `userRef` under a dedicated key (rejected if equal to the encryption key). FIFO write queue, `datasync`, mode `0600`, torn-tail repair, field whitelist. States plainly it is not multi-replica safe and recommends an append-only PostgreSQL table.	Accepted. I chose pseudonymization over storing the raw `userId` because it is client-supplied and can itself be PII.
08	"*Implement POST /secure-inquiry… schema validation… sanitize → encrypt → breaker → audit… 'Service Busy' immediately when OPEN… no leaks of stack/ciphertext/keys… audit failures not silently ignored… integration tests.*"	Thin controller, use-case orchestration, a narrow `AuditFailureReporter` so a failed outcome write is logged, not swallowed. Root-caused step 02: body-parser **accepted UTF-7 JSON** (RFC 8259 requires UTF-8), now 415. Used `res.on('close')` because `req` 'close' fires when the body is read. Proved the tests can fail by temporarily auditing *after* the provider. Real build: 3 × ~2.0 s, then **8 ms**.	Accepted.
09	"*Act as a Senior Backend Engineer performing a strict code review… leaks, AES-GCM mistakes, breaker races, async bugs, missing limits/headers, tests that pass without validating behavior… fix confirmed defects with regression tests… report actual results.*"	Frozen snapshot, four independent read-only reviewers (risk, reliability, resilience, readability) plus a refuter. 0 blocker, 2 critical: **(1)** no security response headers; **(2)** the audit torn-tail repair ran only once per process, so a mid-write `ENOSPC` silently corrupted the **next** record (the existing test could not catch it: its fake wrote nothing before failing). Plus 7 warnings and 7 suggestions.	Fixed both criticals with regression tests that failed first; a separate reviewer approved the diff; 559 tests on two runs. The four confirmed warnings were fixed in step 12.

Step	Your Prompt (What you asked)	AI Response Analysis (What it gave you)	Action Taken
10	"*Prepare for assessment delivery… multi-stage Dockerfile, non-root, Compose with persistent storage, health check, graceful shutdown, no secrets in image… README… secret check… do not claim push/video unless performed.*"	Pinned `node:24.19-alpine` image running as non-root `node` with a healthcheck; Compose with a read-only filesystem, `cap_drop: ALL`, keys required via `${VAR:?}`; README; a dev-key script; secret and PII scans. Verified on real containers: healthy, PII redacted, encrypted audit in the volume, 3 × ~2 s then **22 ms**, `SIGTERM` exit 0. It also caught a wrong example in my review notes.	Accepted; corrected the review record. Committed as 13 Conventional Commits and published to a public GitHub repository.
11	"*Act as a Senior Backend Test Engineer. Create a deterministic end-to-end test… 3 consecutive failures → OPEN… measure the 4th request… provider not called… exact 'Service Busy'… audit policy… recovery after cooldown → CLOSED.*"	Manual clock for the cooldown, counted provider calls, the real 2 s delay only where the provider is actually called. Exposed 2 defects: the fallback was not exactly "Service Busy" (the unit test only checked the **prefix**) and the failure counter was private but **writable at runtime**. Measured: 2052 / 2019 / 2022 ms, then **10 ms** with provider calls still 3, then recovery to CLOSED. Three consecutive green runs.	Accepted both fixes, each with a test that failed first.
12	"*Fix them, then generate a PDF of this format.*" *(original: "arreglalos y después generame un pdf del formato que acabamos de ver")*	Fixed the five open findings, each RED first: **(1)** emails with a local part over 64 characters and non-ASCII/IDN addresses are now fully redacted (14 failures before; it added `\p{M}` beyond the brief so decomposed text and Indic domains do not leak a prefix); **(2)** numeric env values must match `^\d+$` (60 failures before: `" "`, `1e3`, `0x10`, `+5` were accepted); **(3)** unexpected 5xx errors log only type, code, and stack frames, never the message or `cause` (a planted SSN was in the logs before); **(4)** the layer boundary became an import allow-list plus dynamic-import and `globalThis.process` bans (15 violations on probes vs 1 before). It disclosed its trade-offs instead of hiding them.	Accepted; verified independently: 657 tests passed (2 platform-skipped), build clean, five `fix:` / `docs:` commits. Remaining trade-offs documented (Section 4).

3. The "Glass Box" Validation (Deep Logic Check)

Code Block — the circuit breaker's admission and async outcome logic (`src/shared/resilience/consecutive-failures-circuit-breaker.ts`):

```
// Deliberately not async: the fail-fast path must reject without calling or awaiting the operation.
execute<T>(operation: () => Promise<T>): Promise<T> {
  if (!this.admit()) {
    return Promise.reject(new CircuitOpenError());
  }
  return this.run(operation, this.generation);
}

private admit(): boolean {
  switch (this.currentState) {
    case 'CLOSED':
      return true;
    case 'OPEN':
      if (this.now() - this.openedAt < this.cooldownMs) {
        return false;
      }
      this.transitionTo('HALF_OPEN');
      this.probeInFlight = true;
      return true;
    case 'HALF_OPEN':
      if (this.probeInFlight) {
        return false;
      }
      this.probeInFlight = true;
      return true;
  }
}

private async run<T>(operation: () => Promise<T>, admittedIn: number): Promise<T> {
  try {
    const value = await operation();
    this.recordSuccess(admittedIn);
    return value;
  } catch (error) {
    this.recordFailure(admittedIn, error);
    throw error;
  }
}

private recordSuccess(admittedIn: number): void {
  if (admittedIn !== this.generation) {
    return;
  }
  if (this.currentState === 'HALF_OPEN') {
    this.transitionTo('CLOSED');
    return;
  }
  this.failureCount = 0;
}

private recordFailure(admittedIn: number, error: unknown): void {
  if (admittedIn !== this.generation) {
    return;
  }
  if (this.classify(error) === 'ignore') {
    if (this.currentState === 'HALF_OPEN') {
      this.probeInFlight = false;
    }
    return;
  }
  if (this.currentState === 'HALF_OPEN') {
    this.open();
```

```
    return;
  }
  this.failureCount += 1;
  if (this.failureCount >= this.failureThreshold) {
    this.open();
  }
}
```

Your Explanation

1. **Why `execute` is not `async`.** When `admit()` returns `false`, `execute` returns an already-rejected promise *synchronously*: the `operation` function (the 2-second provider call) is never invoked and nothing is awaited. That is why the fourth request answers in ~10 ms.
2. **No race in `admit()`.** Node runs JavaScript on a single thread and `admit()` contains no `await`, so the check `if (this.probeInFlight)` and the assignment `this.probeInFlight = true` run in the same synchronous turn. No other request can be scheduled between them, which guarantees **exactly one** HALF_OPEN probe. OPEN → HALF_OPEN is evaluated lazily, when a request arrives, against an injected clock: no `setTimeout`, nothing leaks, and tests cross the cooldown with `clock.advance(ms)` instead of sleeping.
3. **Where interleaving happens, and how it is neutralized.** In `run()`, `await operation()` yields to the event loop; while the provider takes its 2 s, other requests run `execute()`. When the promise settles, its continuation runs as a microtask and records the outcome. The danger is **stale outcomes**: with five requests admitted in CLOSED, the third failure opens the circuit and the other two fail later. Counted naively, they would push `openedAt` forward (extending the cooldown), and a late success could close an OPEN circuit. The fix is the **generation counter**: each call captures `admittedIn = this.generation` when admitted, every transition increments `generation`, and `recordSuccess`/`recordFailure` discard results from an older generation.
4. **"Consecutive" semantics.** Counting follows completion order. A success in CLOSED resets `failureCount` to 0, so failure, failure, success, failure does **not** open the circuit. The failure that reaches the threshold calls `open()`, which records `openedAt` and transitions to OPEN.
5. **Error propagation.** If `operation()` throws synchronously, the throw happens inside an `async` function and becomes a rejection caught by the same `catch`. After recording, `throw error` rethrows the **same object**: nothing is wrapped or lost. Upstream, `CircuitBreakerAiProvider` maps `CircuitOpenError` to `AiProviderUnavailableError('CIRCUIT_OPEN')`, and the use case's `resolveFallback` turns any provider failure into the `"Service Busy"` fallback. A caller's cancellation is classified `'ignore'` (it neither counts nor resets, and a cancelled probe frees the slot); if the classifier itself throws, the error counts as a failure (fail-safe).
6. **Known limit (documented).** In CLOSED the breaker does not limit concurrency: requests admitted before the transition to OPEN still reach the provider; only those arriving after it fail fast.

4. The Security & Integrity Check (Crucial)

Did the AI commit any of these common sins?

☐ **Hardcoded Secrets** — **not found.** Keys come only from environment variables, validated at startup (canonical base64, exactly 32 bytes, the two keys must differ). Tests generate keys at runtime with `randomBytes(32)`. A scan of every committed file found 0 key literals, and `docker history` contains no key material. *Related exposure found and fixed:* the key was held as a `Buffer` and serialized by `JSON.stringify(config)`; it is now an opaque `KeyObject` (step 04).

☐ **SQL/NoSQL Injection Risk** — **not applicable / not found.** No database. The audit store writes JSON Lines through an explicit field whitelist and reads them through a strict zod schema. Log injection is mitigated by pino's JSON escaping and a validated `X-Request-Id`.

☐ **ReDoS Vulnerability** — **checked, not found.** Matches start only at token boundaries, so the single unbounded quantifier (the email local part, after step 12) still scans linearly; domain labels are bounded; messages are capped at 5,000 characters. Five 100,000-character adversarial inputs run in 8 ms, and extra 100,000-character Unicode inputs in at most 11 ms.

☑ **Error Swallowing** — **found and fixed** (a silent-failure variant). In the audit store, after a write failed mid-line (`ENOSPC`), the torn-tail repair was never re-armed: the **next** write resolved successfully, so the caller believed the record was persisted, but it was glued onto the fragment and lost — silently breaking the "audit before egress" guarantee. The existing test missed it because its fake failed *before* writing anything. Fix: re-arm the repair on every failed write, with a regression test that writes a real 40-byte partial line, then throws `ENOSPC` (it failed before the fix). The same pattern — tests that passed without validating the behavior — was caught four times and tightened each time: an AI-modified test input that masked a UTF-7 bug, a leak test that only searched for base64/hex, a unit test that only checked the prefix of "Service Busy", and this `ENOSPC` fake.

☐ **None of the above**

Detail your fix

Every security or integrity defect found in AI-generated code was fixed with a regression test that failed before the change:

- **Silent audit-record loss** after a failed write (above).
- **Missing security response headers:** added `X-Content-Type-Options: nosniff`, `Cache-Control: no-store`, `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, and `Cross-Origin-Resource-Policy: same-origin` on every response.
- **AES-GCM short-tag acceptance:** without `authTagLength`, Node accepted a forged 4-byte tag; pinned to 16 bytes and validated before decryption.
- **Key serialization leak:** `Buffer` → `KeyObject`, decoded bytes zeroed.
- **UTF-7 JSON accepted:** RFC 8259 requires UTF-8 and UTF-7 can disguise content; now rejected with 415 before the body is read.
- **PII leaks in email redaction:** local parts over 64 characters were not redacted, and non-ASCII addresses leaked a prefix (`müller.hans@…` → `mü<REDACTED: EMAIL>`); both are now fully redacted (step 12).
- **Sensitive data in error logs:** unexpected 5xx errors were logged with their message and `cause`, so a planted SSN reached the logs; they now log only type, code, and stack frames (step 12).
- **Unsafe configuration parsing:** a whitespace-only numeric value became 0 (`AI_PROVIDER_DELAY_MS=" "` silently disabled the delay); numeric variables now require `^\d+$` (step 12).
- **Bypassable layer boundary:** the deny-list allowed other `node:*` modules, dynamic `import()`, and `globalThis.process`; it is now an allow-list of relative imports (step 12).

Residual limitations (documented in the README, not hidden)

- Regex detection is context-blind: 9-digit order numbers are redacted as SSNs and 13–19-digit numbers as cards (no Luhn check, by design: over-redaction over leakage); non-ASCII digits, mixed separators, digits glued to letters, and deliberate obfuscation (zero-width characters, spelled-out values) are not detected.
- In scripts written without spaces, the words around an email are redacted with it (over-redaction, not a leak).
- Mapped 5xx errors (`AUDIT_UNAVAILABLE`) still log their filesystem cause chain (paths and errno, no request content).
- The lint boundary does not catch a relative import of a `src/` root module or an aliased `globalThis`.
- The circuit breaker is in-memory per process and the JSON Lines audit store is single-process: neither is safe for multi-replica deployment (an append-only PostgreSQL table is the documented replacement). No authentication or rate limiting (out of scope).

Final verification: 657 tests passed (2 skipped on Windows, POSIX-only file-mode checks), typecheck, lint, and production build clean; end-to-end circuit test: three failures at ~2 s each, fourth request ~10 ms with the provider not called, recovery to CLOSED after the cooldown.